

SE1020 – Object Oriented Programming

Assignment: Project

Beauty Salon **Appointment Booking System**

Full Project Plan & Component Breakdown

1. Project Overview

The Beauty Salon Appointment Booking System is a web-based application that allows customers to browse salon services, book appointments, and manage their profiles. Salon staff and admins can manage services, track bookings, and handle customer data.

This project is built using Java Spring Boot for the backend, HTML/CSS/JavaScript for the frontend, and uses file-based storage (text files) or a MySQL database for data persistence.

Technology Stack:

- Backend: Java (Spring Boot, JSP Servlets)
 - Frontend: HTML, CSS, JavaScript, Bootstrap or Tailwind CSS
 - Storage: .txt files (items.txt, users.txt, etc.) OR MySQL DB
 - IDE: IntelliJ IDEA
 - Version Control: GitHub
-

2. OOP Concepts You Must Apply

The assignment requires you to demonstrate core OOP principles. Here is how each maps to this project:

OOP Concept	How It Applies in Your Project
Encapsulation	Customer, Service, Appointment classes with private fields and public getters/setters
Inheritance	AdminUser and CustomerUser both inherit from a base User class
Polymorphism	Different appointment confirmation methods for online vs walk-in bookings
Abstraction	Abstract methods for availability checking, payment processing, etc.

3. Project Components (Individual Workload)

Based on the sample workload distribution, your project should be broken into 4–6 components. Below are the recommended components for the Beauty Salon system:

Component 01 – Customer (User) Management

Handles customer registration, login, profile management, and account deletion.

CRUD Operations:

- Create: Register a new customer and save to customers.txt
- Read: Search customer by name or ID
- Update: Edit customer profile (name, phone, email, password)
- Delete: Remove a customer account

UI Pages:

- Customer Registration Page
- Customer Login Page
- Profile Update Page
- Customer List Page (Admin View)

OOP Concepts:

- Encapsulation: Customer class with private fields and getters/setters
- Inheritance: AdminUser and CustomerUser inherit from User
- Polymorphism: Different login/authentication for admin vs customer

File Used: customers.txt

Sample Data Format: C001,Nimal Perera,nimal@email.com,0771234567,Regular

Component 02 – Service Management

Manages the beauty services offered by the salon (haircut, facial, manicure, etc.).

CRUD Operations:

- Create: Add a new service with name, duration, price, and category
- Read: Browse/search services by name or category
- Update: Edit service details (price, duration)
- Delete: Remove discontinued services

UI Pages:

- Service Addition Form (Admin)
- Service List / Browse Page
- Service Edit Page

OOP Concepts:

- Encapsulation: Service class stores service details
- Inheritance: HairService, SkinService, NailService extend Service
- Polymorphism: Different pricing methods for different service types

File Used: services.txt

Sample Data Format: S001,Haircut,Hair,45,1500.00

Component 03 – Appointment Booking Management

The core module — manages booking, viewing, rescheduling, and canceling appointments.

CRUD Operations:

- Create: Book a new appointment (select service, date, time, stylist)
- Read: View upcoming appointments for a customer or by date
- Update: Reschedule an existing appointment
- Delete/Cancel: Cancel an appointment

UI Pages:

- Book Appointment Page
- My Appointments Page (Customer View)
- Appointment Calendar / List (Admin View)
- Reschedule Page

OOP Concepts:

- Encapsulation: Appointment class with booking details
- Abstraction: Method to check time slot availability before booking
- Polymorphism: Different confirmation messages for new bookings vs reschedules

File Used: appointments.txt

Sample Data Format: A001,C001,S001,2025-05-10,10:00,Confirmed

Component 04 – Staff / Stylist Management

Manages the salon's staff members and their assigned services.

CRUD Operations:

- Create: Add a new staff member with name, specialty, and schedule
- Read: View staff profiles and availability
- Update: Edit staff details or working hours
- Delete: Remove a staff member from the system

UI Pages:

- Staff Registration Page
- Staff List Page
- Staff Profile Edit Page

OOP Concepts:

- Encapsulation: Staff class stores name, specialty, schedule
- Inheritance: FullTimeStaff and PartTimeStaff inherit from Staff
- Abstraction: Abstract method for availability calculation

File Used: staff.txt

Sample Data Format: ST001,Kamala,Hair & Nails,Mon-Fri,09:00-17:00

Component 05 – Admin Management

Manages admin accounts and dashboard access.

CRUD Operations:

- Create: Register a new admin account
- Read: View all admins and activity logs

- Update: Modify admin permissions or profile
- Delete: Remove an admin account

UI Pages:

- Admin Dashboard
- Admin Registration Page
- Admin Management Panel

OOP Concepts:

- Encapsulation: Admin class with secure credentials
- Inheritance: AdminUser extends User
- Abstraction: Admin-only operations abstracted from regular users

File Used: admins.txt

Component 06 – Feedback & Review Management

Allows customers to leave reviews for services they have received.

CRUD Operations:

- Create: Submit a review for a completed appointment
- Read: Display reviews for a specific service
- Update: Edit a submitted review
- Delete: Remove a review (customer or admin)

UI Pages:

- Review Submission Page
- View Reviews Page
- Admin Moderation Panel

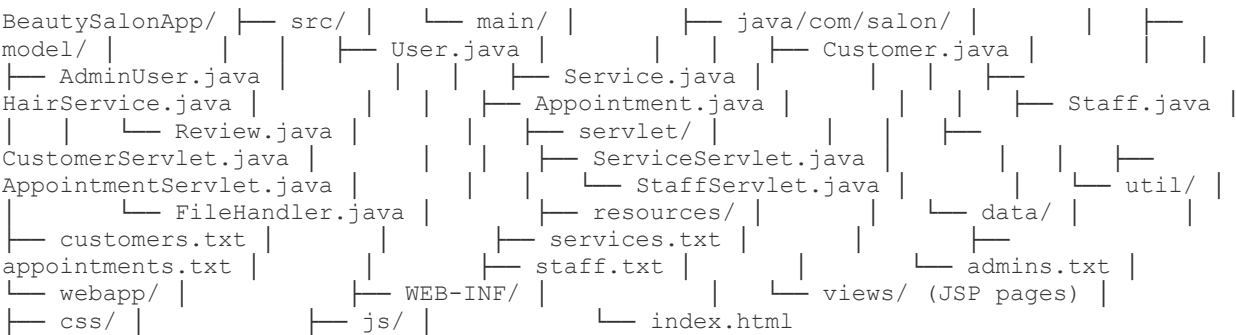
OOP Concepts:

- Encapsulation: Review class stores rating, comment, date
- Inheritance: VerifiedReview and GuestReview extend Review
- Polymorphism: Different display for admin vs customer view

File Used: reviews.txt

4. IntelliJ Project Folder Structure

Here is the recommended folder/file structure for your Spring Boot project in IntelliJ IDEA:



5. Class Diagram Summary

Below is the inheritance hierarchy for your project. You must draw this as a proper UML class diagram for your documentation.

Parent Class	Child Classes	Key Fields
User	CustomerUser, AdminUser	userId, name, email, password
Service	HairService, SkinService, NailService	serviceId, name, price, duration
Staff	FullTimeStaff, PartTimeStaff	staffId, name, specialty, schedule
Review	VerifiedReview, GuestReview	reviewId, rating, comment, date

6. File Handling (Data Storage)

Instead of a database, you will use text files. Each file stores records in CSV format (comma-separated values). Here is how each file works:

File Name	Stores	Sample Line
customers.txt	Customer records	C001,Nimal,nimal@mail.com,0771234567,Regular
services.txt	Salon services	S001,Haircut,Hair,45,1500.00
appointments.txt	Bookings	A001,C001,S001,2025-05-10,10:00,Confirmed

staff.txt	Stylist info	ST001,Kamala,Hair,Mon-Fri,09:00-17:00
admins.txt	Admin accounts	ADM001,admin,admin@salon.com,hashed_pass
reviews.txt	Customer reviews	R001,C001,S001,5,Great service!,2025-05-10

Your FileHandler.java utility class will have methods like readAll(), writeRecord(), updateRecord(), and deleteRecord() that all servlets use for file I/O operations.

7. Minimum UI Pages Required

The assignment requires a minimum of 3 UIs per component. Across all components, here are the key pages your system needs:

Page	Component	Type
Home / Landing Page	General	Public
Customer Registration	Customer Mgmt	Public
Customer Login	Customer Mgmt	Public
Profile Update	Customer Mgmt	Customer
Browse Services	Service Mgmt	Public
Book Appointment	Appointment	Customer
My Appointments	Appointment	Customer
Admin Dashboard	Admin	Admin
Manage Services	Service Mgmt	Admin
Manage Appointments	Appointment	Admin
Staff List	Staff Mgmt	Admin
Submit Review	Feedback	Customer

8. Step-by-Step Development Plan

Follow these steps to build your project from scratch:

Phase 1 – Setup (Week 1)

1. Create a new Spring Boot project in IntelliJ IDEA

2. Set up the project folder structure as shown in Section 4
3. Create a GitHub repository and make your first commit
4. Add Bootstrap or Tailwind CSS to your project

Phase 2 – Model Classes (Week 1–2)

5. Write User.java (base class) with all fields, constructor, getters/setters
6. Write CustomerUser.java and AdminUser.java extending User
7. Write Service.java and subclasses (HairService, etc.)
8. Write Appointment.java, Staff.java, Review.java

Phase 3 – File Handling Utility (Week 2)

9. Write FileHandler.java with readAll(), writeRecord(), updateRecord(), deleteRecord()
10. Test file reading and writing with sample data

Phase 4 – Servlets & Backend Logic (Week 2–3)

11. Write CustomerServlet.java handling register, search, update, delete
12. Write ServiceServlet.java for service CRUD
13. Write AppointmentServlet.java for booking logic
14. Write StaffServlet.java for staff management

Phase 5 – JSP Pages / UI (Week 3–4)

15. Build all HTML/JSP pages for each component
16. Connect forms to servlets using action URLs
17. Style pages with Bootstrap/Tailwind

Phase 6 – Testing & Git History (Week 4)

18. Test all CRUD operations with sample data files
19. Fix bugs and ensure data saves/loads correctly
20. Make regular git commits to build a visible commit history
21. Prepare class diagrams and final report

9. Deliverables Checklist

Deliverable	What to Submit	Status
-------------	----------------	--------

Source Code	All .java, .jsp, config files on GitHub	☐ To Do
Class Diagrams	UML diagram showing inheritance hierarchy	☐ To Do
Final Report	Report including Git commit history screenshots	☐ To Do
Data Files	customers.txt, services.txt, appointments.txt etc.	☐ To Do
Viva Presentation	Live demo + explain your component's Java code	☐ To Do
Git History	Visible commits from each team member	☐ To Do

10. Viva Preparation Tips

During the viva, the lecturer will focus on the backend Java code. Be prepared to explain:

- How encapsulation is applied in your model classes (private fields, getters/setters)
- How inheritance works in your User → CustomerUser/AdminUser hierarchy
- How polymorphism is demonstrated (method overriding, different behaviors)
- How your FileHandler reads and writes data to .txt files
- How your Servlet processes HTTP requests (doGet, doPost methods)
- Why you made specific design decisions in your component

Also prepare to show your Git repository with commit history, and walk through the UI pages demonstrating each CRUD operation live.